

Performance Testing

Date	01-07-2026
Team ID	SWTID-2026-3845
Project Name	SMART LENDER
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman
Type of Testing	Functional Testing, Load Testing
Target Module / API	/predict endpoint — loan approval prediction API
Test Environment	Local
Test Date	02-07-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Low-risk applicant submits valid details	1	5	Application predicts "Approved" with high confidence
2	High-risk applicant submits details	1	5	Application predicts "Rejected" or flags for further review
3	Submit form with missing/invalid fields	1	5	Application returns validation error instead of crashing
4	Multiple concurrent users submit prediction requests simultaneously	10	30	Application handles concurrent requests without errors

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.8 seconds	Pass	Model prediction returned quickly under normal load
2	Response Time (Max)	< 5 seconds	1.6 seconds	Pass	Slight increase observed under concurrent request
3	Throughput (Req/sec)	10 req/sec	12 req/sec	Pass	Exceeded target during 10-user concurrent test
4	Error Rate	< 1%	0%	Pass	No failed request during testing
5	CPU Utilization	< 80%	45%	Pass	Local machine handled load comfortably
6	Memory Utilization	< 80%	52%	Pass	Model and dependencies loaded

Step 4: Observations & Analysis

Key Findings:

The Flask application performed well under normal and moderate concurrent load, with average response times well within the target of 2 seconds. The XGBoost model returned predictions quickly, confirming the system is suitable for real-time loan approval decisions. Error rates remained at 0% across all test scenarios, indicating stable request handling.

Bottlenecks Identified:

A slight increase in response time was observed when handling 10 concurrent requests simultaneously, likely due to the model being reloaded into memory on each request rather than being cached. Since the application currently runs on a single local instance, there is no load balancing to distribute concurrent traffic.

Optimization Steps Taken:

The trained model was loaded once at application startup instead of on every request, reducing repeated overhead. Input validation was added to the prediction endpoint to catch malformed requests early and avoid unnecessary processing. Basic error handling was implemented around the prediction pipeline to prevent crashes on invalid inputs.

